```
%% --- Load emission dataset and filter for UK ---
dataset = readtable('GCB2024v18_MtCO2_flat.csv');
```

```
Warning: Column headers from the file were modified to make them valid MATLAB identifiers
before creating variable names for the table. The original column headers are saved in the
VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as table variable names.
```

```
dataset = dataset(strcmp(dataset.Country, "United Kingdom"), :);
head(dataset,5);
```

Country	ISO3166_1Alpha_3	UNM49	Year	Total	Coal	Oil	Gas	Cement
{'United Kingdom'}	{'GBR'}	826	1750	9.3059	9.3059	0	0	NaN
{'United Kingdom'}	{'GBR'}	826	1751	9.4072	9.4072	0	0	NaN
{'United Kingdom'}	{'GBR'}	826	1752	9.5052	9.5052	0	0	NaN
{'United Kingdom'}	{'GBR'}	826	1753	9.6105	9.6105	0	0	NaN
{'United Kingdom'}	{'GBR'}	826	1754	9.7336	9.7336	0	0	NaN

```
%% --- Linearly Interpolate any missing values and recompute total ---
vars = {'Coal','Oil','Gas','Cement','Flaring','Other'};

for v = vars
    x = dataset.(v{1});
    i0 = find(~isnan(x),1,'first');
    x(i0:end) = fillmissing(x(i0:end),'linear');
    x(isnan(x)) = 0;              % ensure no NaNs remain
    dataset.(v{1}) = x;
end

dataset.Total = dataset.Coal + dataset.Oil + dataset.Gas + ...
                dataset.Cement + dataset.Flaring + dataset.Other;

data = dataset(:, {'Year','Total'});
```

```
%% --- Load population ---
population_dataset = readtable('GCB2024v18_population_flat.csv');
population_dataset =
population_dataset(strcmp(population_dataset.CountryName, "United
Kingdom"), :);
population_dataset = population_dataset(:, {'Year','Population'});
```

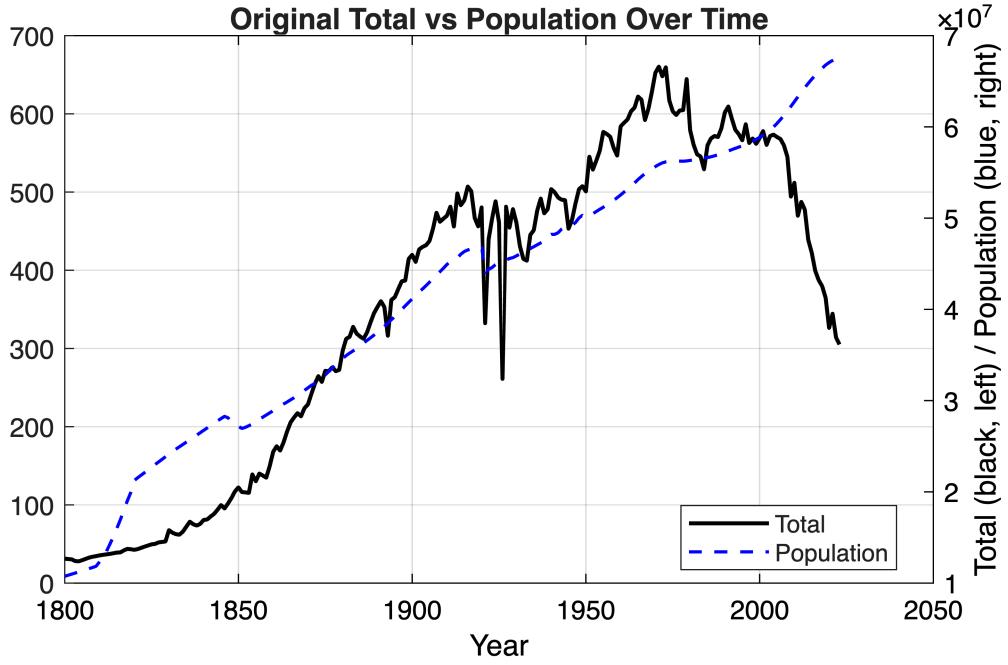
```
%% --- Compute population for years before population dataset starts ---
y0 = min(population_dataset.Year);

idx = dataset.Year < y0 & ~isnan(dataset.PerCapita);

pop_missing = table();
pop_missing.Year = dataset.Year(idx);
pop_missing.Population = dataset.Total(idx) ./ dataset.PerCapita(idx);
```

```
pop_missing.Population = pop_missing.Population * 1e6;

%% --- Combine with population dataset
population_dataset = [pop_missing; population_dataset];
head(population_dataset, 5);
```

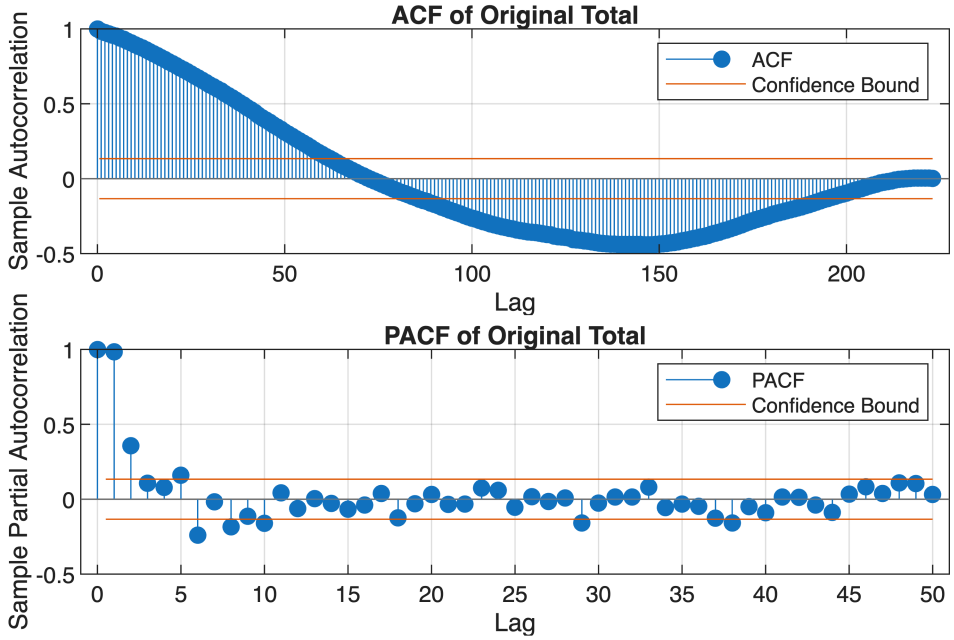
Year	Population
1800	1.075e+07
1801	1.0866e+07
1802	1.0984e+07
1803	1.1102e+07
1804	1.1222e+07

```
%% --- Align population data with emmission data ---
commonYears = intersect(data.Year, population.Year);
data = data(ismember(data.Year, commonYears), :);
population = population(ismember(population.Year, commonYears), :);
merged = join(data, population, 'Keys', 'Year');
```

```
%% --- Visualize original data ---
figure('Color','w');
plot(data.Year, data.Total, 'k-', 'LineWidth', 1.5); hold on;
yyaxis right
plot(population.Year, population.Population, 'b--', 'LineWidth', 1.2);
xlabel('Year');
ylabel('Total (black, left) / Population (blue, right)');
title('Original Total vs Population Over Time');
legend('Total','Population','Location','best');
grid on; box on;
set(gca,'Color','w','XColor','k','YColor','k');
```

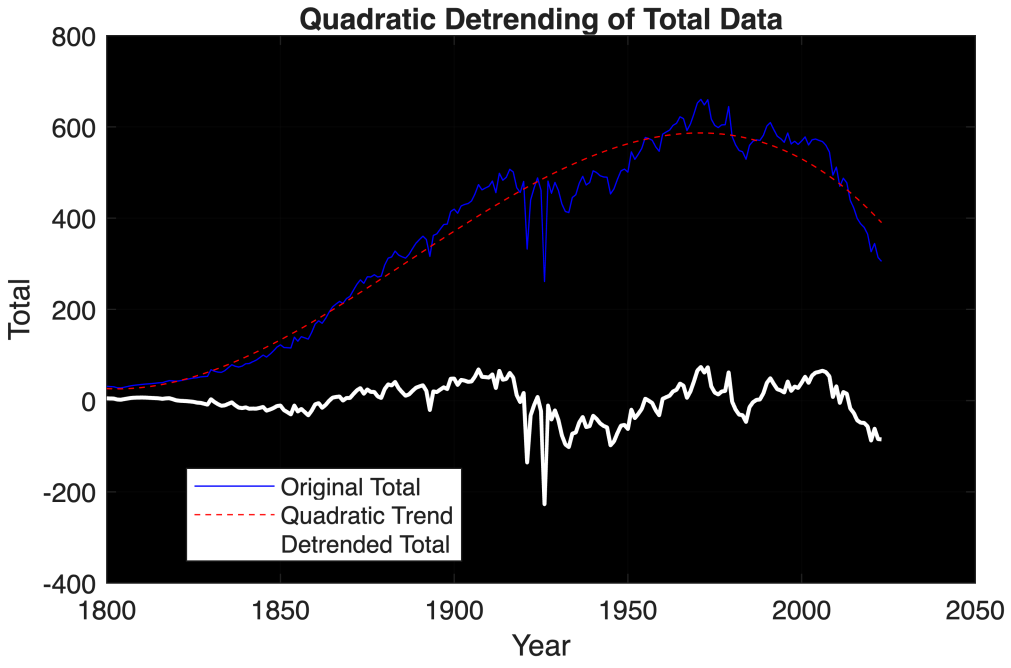



```
%% --- ACF and PACF of Original Total Data ---
figure;

subplot(2,1,1);
autocorr(merged.Total, 'NumLags', 223);
title('ACF of Original Total');

subplot(2,1,2);
parcorr(merged.Total, 'NumLags', 50);
title('PACF of Original Total');
```



```
%% --- Detrend Total with cubic polynomial (centered years) ---
yrs = merged.Year;
yrs_c = yrs - mean(yrs);                    % center years
p = polyfit(yrs_c, merged.Total, 3);        % cubic fit
trend = polyval(p, yrs_c);
merged.Total_detrended = merged.Total - trend;
```

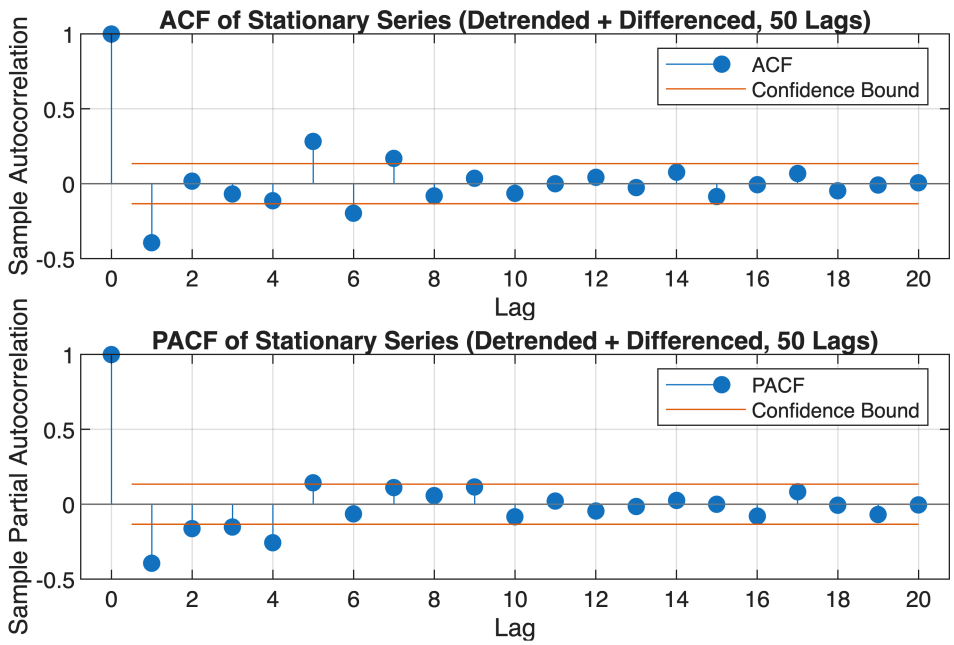
```
%% --- Show cubic trend ---
figure;
plot(yrs, merged.Total, 'b', 'DisplayName', 'Original Total');
hold on;
plot(yrs, trend, 'r--', 'DisplayName', 'Quadratic Trend');
plot(yrs, merged.Total_detrended, 'w', 'LineWidth', 1.5, 'DisplayName',
'Detrended Total');
xlabel('Year');
ylabel('Total');
title('Quadratic Detrending of Total Data');
legend('Location', 'best');
grid on;
set(gca, 'Color', 'k'); % optional: black background for visibility
hold off;
```



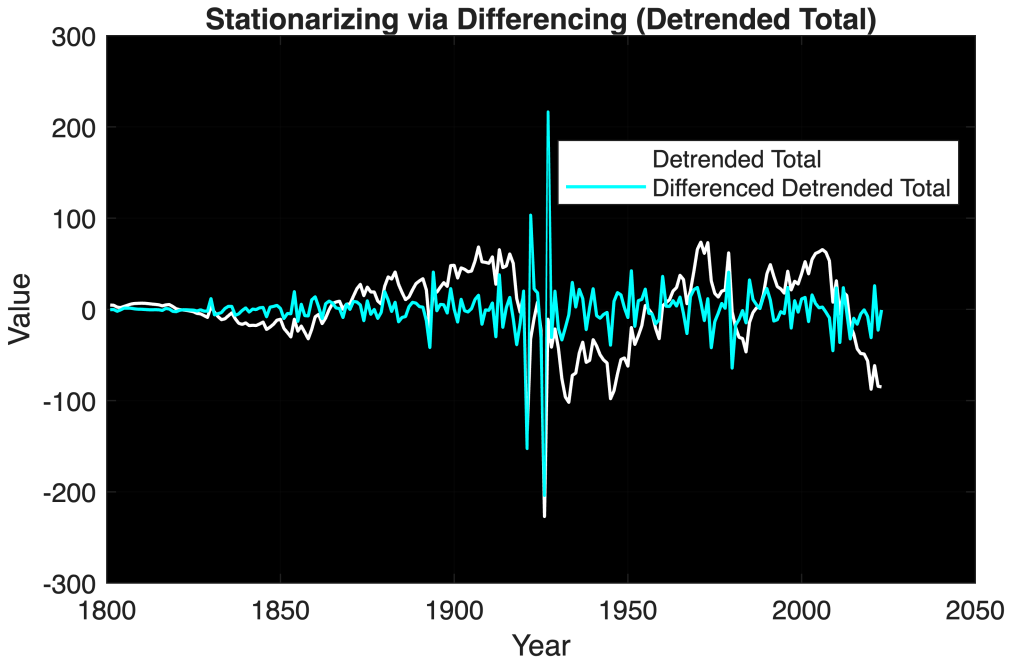

```
%% --- Difference detrended Total to stationarize ---
merged.Total_diff = [NaN; diff(merged.Total_detrended)];
merged = merged(2:end,:); % drop first NaN
```

```
%% --- ACF and PACF of Stationary Series (More Lags) ---
```

```
figure;

subplot(2,1,1);
autocorr(merged.Total_diff, 'NumLags', 20);
title('ACF of Stationary Series (Detrended + Differenced, 50 Lags)');

subplot(2,1,2);
parcorr(merged.Total_diff, 'NumLags', 20);
title('PACF of Stationary Series (Detrended + Differenced, 50 Lags)');
```




```
%% --- Print graphs ---
yrs = yrs(2:end);  % keep years consistent with merged after dropping first
row

figure;
plot(yrs, merged.Total_detrended, 'w', 'LineWidth', 1.2, 'DisplayName',
'Detrended Total');
hold on;
plot(yrs, merged.Total_diff, 'c', 'LineWidth', 1.2, 'DisplayName',
'Differenced Detrended Total');
xlabel('Year');
ylabel('Value');
title('Stationarizing via Differencing (Detrended Total)');
legend('Location', 'best');
grid on;
set(gca, 'Color', 'k');
hold off;
```




```
%% --- Create timetable for ARIMAX ---
TT = table2timetable(merged, 'RowTimes', datetime(merged.Year,1,1));
```

```
%% --- Define splits ---
train = TT(TT.Year <= 2013, :);
valid = TT(TT.Year > 2013 & TT.Year <= 2018, :);
train_valid = TT(TT.Year <= 2018, :);
test  = TT(TT.Year > 2018 & TT.Year <= 2023, :);
```

```
%% --- Grid search (inner validation) ---
bestRMSE = Inf; bestParams = [NaN NaN NaN]; results = [];
for p_ = 0:6
  for q_ = 0:6
    try
      mdl = regARIMA(p_, 0, q_);
      fit = estimate(mdl, train.Total_diff, 'X', train.Population,
'Display', 'off');

      nSteps = height(valid);
      [yF, ~] = forecast(fit, nSteps, 'Y0', train.Total_diff, ...
                         'X0', train.Population, 'XF', valid.Population);

      rmse = sqrt(mean((valid.Total_diff - yF).^2, 'omitnan'));
      mae  = mean(abs(valid.Total_diff - yF), 'omitnan');

      fprintf('regARIMA(%d,0,%d)+X => Valid RMSE=%.3f | MAE=%.3f\n', p_,
q_, rmse, mae);
```

```
      results = [results; p_ q_ rmse];
      if rmse < bestRMSE
        bestRMSE = rmse; bestParams = [p_ 0 q_];
        bestMAE = mae; bestModel = fit;
      end
    catch ME
      fprintf('regARIMA(%d,0,%d)+X => FAILED (%s)\n', p_, q_, ME.message);
      results = [results; p_ q_ NaN];
    end
  end
end
```

```
regARIMA(0,0,0)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(0,0,1)+X => Valid RMSE=17.271 | MAE=13.065
regARIMA(0,0,2)+X => Valid RMSE=17.095 | MAE=12.973
regARIMA(0,0,3)+X => Valid RMSE=20.354 | MAE=14.796
regARIMA(0,0,4)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(0,0,5)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(0,0,6)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(1,0,0)+X => Valid RMSE=17.609 | MAE=13.170
regARIMA(1,0,1)+X => Valid RMSE=17.113 | MAE=12.981
regARIMA(1,0,2)+X => Valid RMSE=17.095 | MAE=12.973
regARIMA(1,0,3)+X => Valid RMSE=20.354 | MAE=14.796
regARIMA(1,0,4)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(1,0,5)+X => Valid RMSE=18.212 | MAE=13.360
regARIMA(1,0,6)+X => Valid RMSE=18.335 | MAE=13.413
regARIMA(2,0,0)+X => Valid RMSE=16.330 | MAE=12.759
regARIMA(2,0,1)+X => Valid RMSE=18.698 | MAE=14.313
regARIMA(2,0,2)+X => Valid RMSE=17.095 | MAE=12.973
regARIMA(2,0,3)+X => Valid RMSE=20.354 | MAE=14.796
regARIMA(2,0,4)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(2,0,5)+X => Valid RMSE=18.847 | MAE=13.589
regARIMA(2,0,6)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(3,0,0)+X => Valid RMSE=17.969 | MAE=12.986
regARIMA(3,0,1)+X => Valid RMSE=17.271 | MAE=13.065
regARIMA(3,0,2)+X => Valid RMSE=17.095 | MAE=12.973
regARIMA(3,0,3)+X => Valid RMSE=20.354 | MAE=14.796
regARIMA(3,0,4)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(3,0,5)+X => Valid RMSE=18.165 | MAE=16.216
regARIMA(3,0,6)+X => Valid RMSE=17.424 | MAE=15.148
regARIMA(4,0,0)+X => Valid RMSE=16.614 | MAE=12.894
regARIMA(4,0,1)+X => Valid RMSE=16.838 | MAE=14.095
regARIMA(4,0,2)+X => Valid RMSE=18.484 | MAE=15.825
regARIMA(4,0,3)+X => Valid RMSE=18.534 | MAE=15.888
regARIMA(4,0,4)+X => Valid RMSE=16.480 | MAE=13.575
regARIMA(4,0,5)+X => Valid RMSE=17.390 | MAE=15.235
regARIMA(4,0,6)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(5,0,0)+X => Valid RMSE=16.020 | MAE=13.243
regARIMA(5,0,1)+X => Valid RMSE=18.524 | MAE=16.058
regARIMA(5,0,2)+X => Valid RMSE=18.573 | MAE=15.918
regARIMA(5,0,3)+X => Valid RMSE=20.354 | MAE=14.796
regARIMA(5,0,4)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(5,0,5)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(5,0,6)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(6,0,0)+X => Valid RMSE=18.144 | MAE=15.020
regARIMA(6,0,1)+X => Valid RMSE=18.422 | MAE=15.781
regARIMA(6,0,2)+X => Valid RMSE=17.095 | MAE=12.973
regARIMA(6,0,3)+X => Valid RMSE=20.354 | MAE=14.796
regARIMA(6,0,4)+X => Valid RMSE=17.055 | MAE=12.952
regARIMA(6,0,5)+X => Valid RMSE=17.164 | MAE=14.518
regARIMA(6,0,6)+X => Valid RMSE=17.642 | MAE=15.141
```

```
fprintf('\nBest regARIMA(%d,%d,%d)+Population: Valid RMSE=%.3f |
MAE=%.3f\n', ...
  bestParams(1), bestParams(2), bestParams(3), bestRMSE, bestMAE);
```

```
Best regARIMA(5,0,0)+Population: Valid RMSE=16.020 | MAE=13.243
```

```
%% --- Retrain best model on all ≤2018 data ---
finalModel = regARIMA(bestParams(1), bestParams(2), bestParams(3));
finalFit = estimate(finalModel, train_valid.Total_diff, 'X',
train_valid.Population, 'Display', 'off');
```

```
%% --- Forecast test (2019–2023) ---
nStepsTest = height(test);
[yPredTest, ~] = forecast(finalFit, nStepsTest, ...
    'Y0', train_valid.Total_diff, ...
    'X0', train_valid.Population, ...
    'XF', test.Population);
```

```
%% --- Reconstruct to original scale (corrected) ---
yrs_mean = mean(yrs);
trend_test = polyval(p, test.Year - yrs_mean);

base_detr = merged.Total_detrended(merged.Year == 2018);
yPredTest_detr = base_detr + cumsum(yPredTest);

% Add back cubic trend to get back to original scale
yPredTest_orig = yPredTest_detr + trend_test;
```

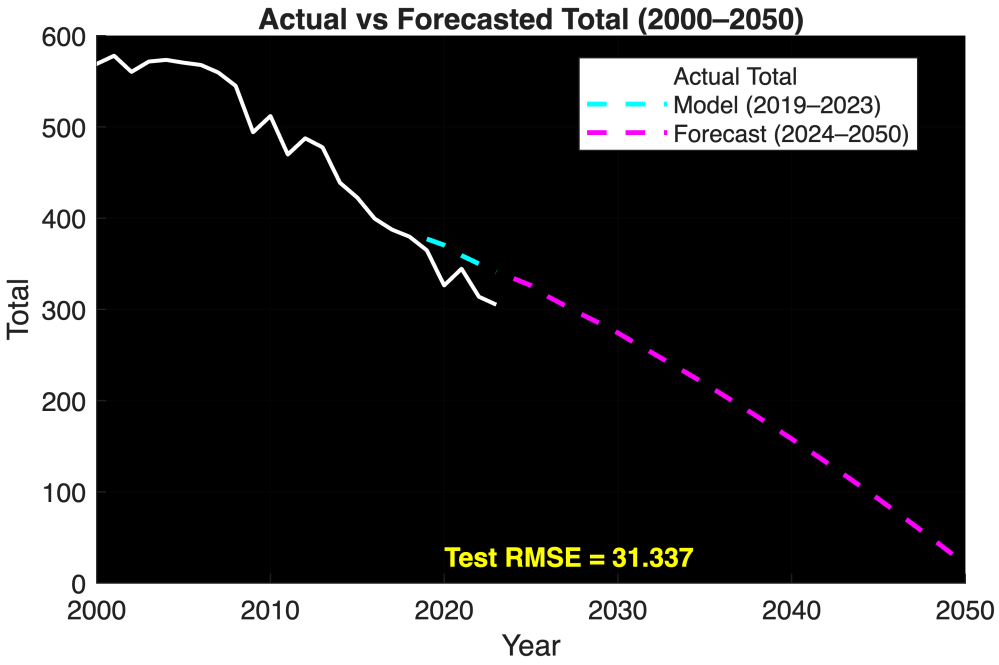
```
%% --- Compute RMSE on original-scale test values ---
trueTest = merged.Total(ismember(merged.Year, test.Year));
testRMSE_orig = sqrt(mean((trueTest - yPredTest_orig).^2, 'omitnan'));
fprintf('Test (2019–2023, Original Scale) RMSE=%.3f\n', testRMSE_orig);
```

```
Test (2019–2023, Original Scale) RMSE=31.337
```

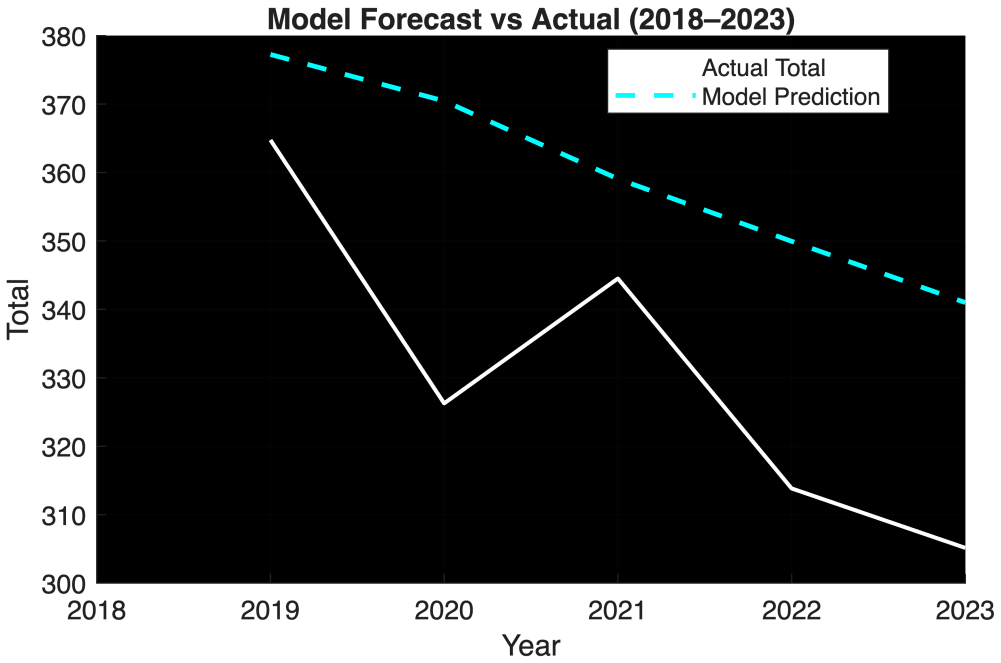
```
%% --- Long-term forecast (2024–2050) ---
futureYears = (2024:2050)';
nStepsFuture = numel(futureYears);

[yPredFuture, ~] = forecast(finalFit, nStepsFuture, ...
    'Y0', train_valid.Total_diff, ...
    'X0', train_valid.Population, ...
    'XF', repelem(test.Population(end), nStepsFuture, 1));  % assume
constant population

trend_future = polyval(p, futureYears - yrs_mean);
```

```
yPredFuture_detr = yPredTest_detr(end) + cumsum(yPredFuture);
yPredFuture_orig = yPredFuture_detr + trend_future;
```

```
%% --- Plot 2000–2050 with test RMSE and forecast ---
figure;
hold on;

% Actual data (2000–2023)
plot(merged.Year, merged.Total, 'w', 'LineWidth', 1.5, 'DisplayName',
'Actual Total');

% Model predictions (2019–2023 test)
plot(test.Year, yPredTest_orig, 'c--', 'LineWidth', 1.8, 'DisplayName',
'Model (2019–2023)');

% Long-term forecast (2024–2050)
plot(futureYears, yPredFuture_orig, 'm--', 'LineWidth', 1.8, 'DisplayName',
'Forecast (2024–2050)');

% RMSE annotation
text(2020, min(merged.Total)*1.05, ...
    sprintf('Test RMSE = %.3f', testRMSE_orig), ...
    'Color', 'y', 'FontSize', 10, 'FontWeight', 'bold');

xlabel('Year');
ylabel('Total');
title('Actual vs Forecasted Total (2000–2050)');
xlim([2000 2050]);    % <-- limit visible years
legend('Location', 'best');
grid on;
set(gca, 'Color', 'k');
hold off;
```




```
%% --- Plot Zoomed-In Test Period (2018–2023) ---
figure;
hold on;

% Actual Total for 2018–2023
plot(test.Year, test.Total, 'w', 'LineWidth', 1.5, 'DisplayName', 'Actual
Total');

% Model Predictions for 2019–2023
plot(test.Year, yPredTest_orig, 'c--', 'LineWidth', 1.8, 'DisplayName',
'Model Prediction');

xlabel('Year');
ylabel('Total');
title('Model Forecast vs Actual (2018–2023)');
legend('Location', 'best');
xlim([2018 2023]);
grid on;
set(gca, 'Color', 'k');
hold off;
```




```
%% --- Actual vs Model-Predicted Total (1800–2023) with RMSE ---

% 1. Get in-sample fitted differenced values
yfitted = infer(finalFit, train_valid.Total_diff, 'X',
train_valid.Population);

% 2. Reconstruct fitted (detrended) levels
base_detr = train_valid.Total_detrended(1);
yfit_detr = base_detr + cumsum(yfitted);

% 3. Add back trend to return to original scale
yrs_mean = mean(yrs);
trend_fit = polyval(p, train_valid.Year - yrs_mean);
yfit_orig = yfit_detr + trend_fit;

% 4. Compute RMSE across full 1800–2023 span (train + test)
trueAll = merged.Total(merged.Year <= 2023);
predAll = [yfit_orig; yPredTest_orig];
yearAll = [train_valid.Year; test.Year];
overallRMSE = sqrt(mean((trueAll - predAll).^2, 'omitnan'));

% 5. Plot actual vs model prediction
figure;
hold on;
plot(merged.Year, merged.Total, 'w', 'LineWidth', 1.5, 'DisplayName',
'Actual Total');
plot(yearAll, predAll, 'c--', 'LineWidth', 1.8, 'DisplayName', 'Model
Prediction');
xlabel('Year');
```

```
ylabel('Total');
title(sprintf('Actual vs Model-Predicted Total (1800–2023) — RMSE = %.3f',
overallRMSE));
legend('Location', 'best');
grid on;
set(gca, 'Color', 'k');
hold off;
```